

APPNet: Automatic Feature Partitioning-Based Parameter Personalized Network for Conversion Prediction in E-commerce

Shui Liu
shui.lius@alibaba-inc.com
Alibaba Group
Hangzhou, Zhejiang, China

Mingyuan Tao
mingyuan.tmy@alibaba-inc.com
Alibaba Group
Hangzhou, Zhejiang, China

Maofei Que
maofei.qmf@alibaba-inc.com
Alibaba Group
Hangzhou, Zhejiang, China

Pan Li
pan.li@scheller.gatech.edu
Georgia Tech
Atlanta, Georgia, USA

Zhuoran Zhuang
nantian@alibaba-inc.com
Alibaba Group
Hangzhou, Zhejiang, China

Abstract

Traditional conversion rate prediction models suffer from suboptimal performance due to sharing the same network parameters for all instances, failing to capture heterogeneous underlying distributions across instances. Recent parameter personalization models address this issue by grouping instances based on selected features, then generating personalized parameters for each group. However, existing parameter personalization models face challenges: (1) Taking prior information features as grouping condition for model parameter personalization leads to suboptimal performance due to human's limited understanding of data distribution. (2) Using all the features for both parameter generation module and deep neural network (DNN) of conversion prediction tasks causes gradient conflicts during backpropagation. A better approach is to automatically select features as grouping condition based on data distribution through iterative learning. Therefore, we propose Automatic Feature Partitioning-Based Parameter Personalized Network (APPNet), which consists of two components: Automatic Feature Partitioning (AFP) and Parameter Personalized Network (PPNet). The AFP module automatically partitions all the features into two parts: one part for DNN of conversion prediction tasks, and the other part for PPNet module to generate weights to adjust DNN parameters of conversion prediction tasks. Specifically, we implemented two versions of AFP: feature-wise AFP and bit-wise AFP. The feature-wise AFP partitions features at the feature field granularity, while the bit-wise AFP partitions each bit of the feature embeddings. The PPNet module adjusts model parameters of conversion prediction task for each group of instances by applying element-wise multiplication to the DNN parameters of conversion tasks. Extensive offline experiments demonstrate APPNet outperforms previous parameter personalized models. Furthermore, online A/B testing in production system achieved a 1.09% improvement on conversion rate, validating its practical effectiveness.

CCS Concepts

• **Information systems** → **Personalization; Recommender systems**; • **Computing methodologies** → **Neural networks**.

Keywords

Parameter Personalized Network, Straight-Through Estimator, Search & Ranking

ACM Reference Format:

Shui Liu, Mingyuan Tao, Maofei Que, Pan Li, and Zhuoran Zhuang. 2026. APPNet: Automatic Feature Partitioning-Based Parameter Personalized Network for Conversion Prediction in E-commerce. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining (WSDM '26)*, February 22–26, 2026, Boise, ID, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3773966.3777922>

1 Introduction

Deep neural network (DNN) models are widely used for click-through rate (CTR) and click-through rate & post-click conversion rate (CTCVR) prediction in e-commerce search and recommendation systems. However, traditional DNN models share the same DNN parameters for all instances. Due to inconsistent distributions among different instance groups, such a manner can hardly characterize each of the instances which may have different underlying distributions, leading to limited performance of DNN models [21].

In recent years, parameter-personalized networks have been proposed to improve this issue. APG [21] proposes to dynamically generate personalized network parameters for different instances by using the DNN inputs as conditions, combined with over-parameterization to further optimize the generation efficiency. However, it faces gradient conflict problems. As shown in the left part of Figure 1, all the features are used both as inputs for parameter generation module and as inputs for the DNN network, facing gradient conflict issues in gradient backpropagation stage. PPNet [4], as shown in the middle part of Figure 1, proposes to take prior information features and other features (without gradient backpropagation) as inputs to obtain personalized weights to adjust DNN parameters. Allocating more features to DNN and fewer to PPNet improves DNN performance but reduces PPNet's personalization strength. Conversely, the opposite allocation has the reverse effect. Therefore we argue that manually selecting grouping conditions based on prior knowledge or human intervention is inefficient and



This work is licensed under a Creative Commons Attribution 4.0 International License. WSDM '26, Boise, ID, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2292-9/2026/02
<https://doi.org/10.1145/3773966.3777922>

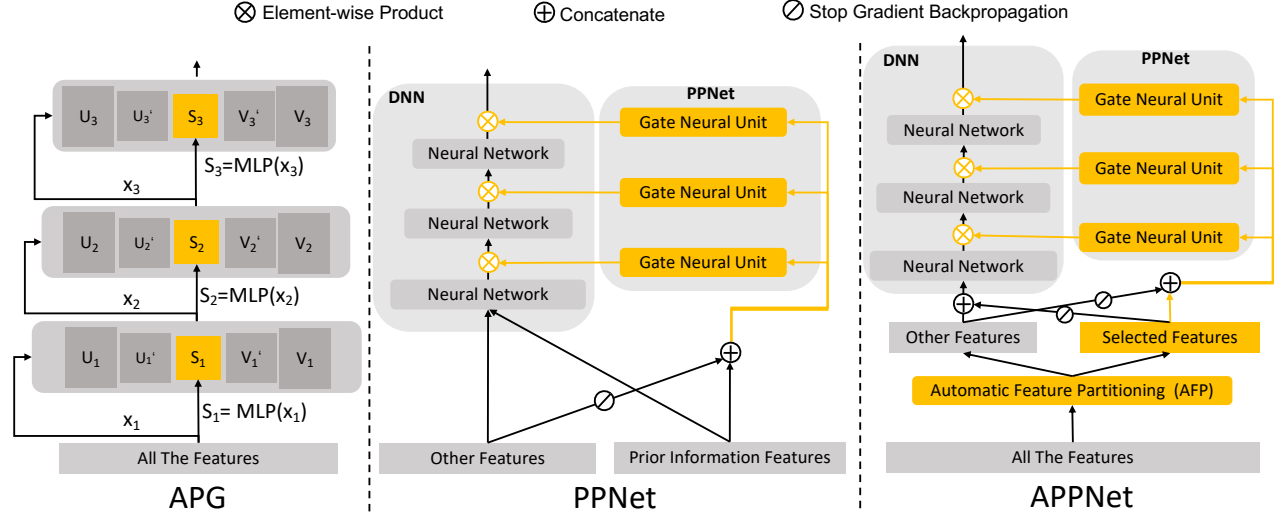


Figure 1: Overview of model architectures of our proposed APPNet and its competitors: APG (left), PPNet (middle), APPNet (right)

suboptimal due to humans' limited understanding of data distribution. A better approach is to automatically partition features into two parts (one part for DNN of conversion rate prediction tasks, and the other part for PPNet as grouping features) based on data distribution through iterative learning.

To automatically partition features into two subsets, a straightforward approach is to use binary neurons to classify features into two categories and iteratively optimize the partitioning strategy to identify the optimal feature division. However, binary neurons face the problem of non-differentiable gradients due to their discrete nature, making gradient backpropagation infeasible. Through investigation, we found that the Straight-Through Estimator (STE)[2] make it possible to uses discrete values during forward propagation stage and continuous values during backpropagation stage. This inspired us to combine STE with gradient stopping backpropagation strategies to implement an automatic feature partitioning module. Specifically, we proposed an Automatic Feature Partitioning-Based Parameter Personalized Network (APPNet) model, as shown in the right part of Figure 1, which consists of two components: the Automatic Feature Partitioning (AFP) module and Parameter Personalized Network (PPNet). Through the AFP module, all features are automatically divided into two parts: one part as the input for the network parameter-personalized network, and the other part as the input for DNN of the conversion rate prediction task. The design of AFP module avoids gradient conflicts between the input features of DNN and the input features of PPNet by using gradient stopping strategy, and learns to select instance grouping features from data distributions, thereby improving model performance. We implement two versions of AFP: feature-wise and bit-wise. The feature-wise AFP partitions features at the feature field granularity, while the bit-wise AFP partitions each bit of the feature embeddings. The PPNet, which consists of multiple layers of Gate Neural Unit (GNU), takes the grouping features selected by the AFP module as

input and outputs scaling weights to adjust DNN network parameters of the conversion prediction task, thereby achieving parameter personalization.

Our contributions can be summarized as follows:

- (1) We proposed a novel APPNet model that uses the automatic feature partitioning module to select appropriate features to group instances, then uses PPNet to adjust DNN parameters of conversion rate prediction tasks for different groups, ultimately enhance the model's capability to characterize samples with different distributions.
- (2) Through extensive experiments, we demonstrate the effectiveness of the model and its components, and deployed it to the production environment for online A/B testing, achieving an improvement of 1.09% on CTCVR.

2 Related Work

2.1 Parameter Personalized Network

Traditional Models suffer sub-optimal performance due to sharing the same model parameters between all instances. Learning Hidden Unit Contributions (LHUC) [14] takes user ID as the personalized identifiers to learn speaker-specific hidden unit contributions to improve the accuracy of speech recognition for different speakers by scaling the model's hidden layers with personalized contributions. Personalized cOld Start MOdules (POSO) [6] introduces user-group-specialized sub-modules to reinforce the personalization of existing modules. Specifically, the POSO takes whether a user is new and user exposure counts as input features, and uses a gate layer to output scaling scalars that adjust network parameters. This enables new and regular users to utilize different network parameters to achieve personalized model responses. Adaptive Parameter Generation Network (APG) [21] dynamically generates parameters for deep CTR (Click-Through Rate) models on-the-fly based on different instances. The network parameters of the APG are generated by

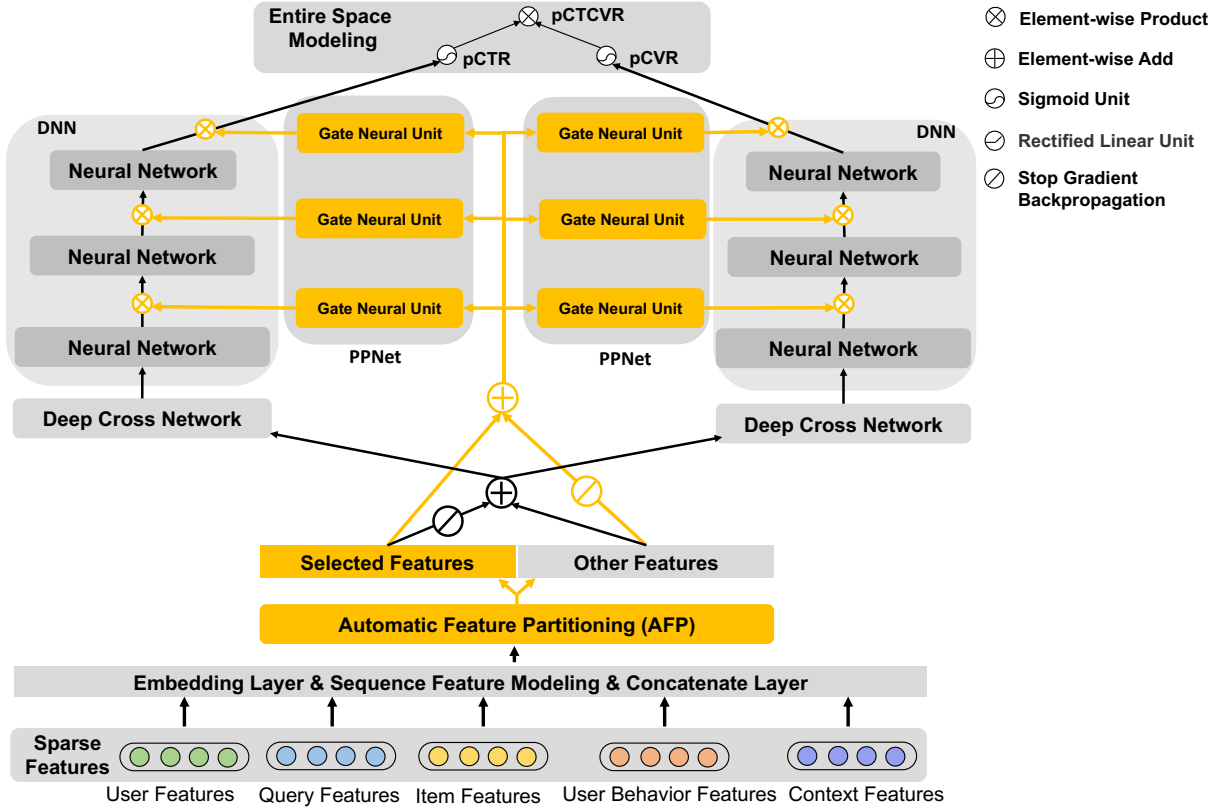


Figure 2: The overview of proposed Automatic feature partitioning-based Parameter Personalized Network (APPNet), which consists of Embedding Layer, Sequence Feature Modeling, Concatenate Layer, Automatic Feature Partitioning (AFP), PPNet, Deep Cross Network (DCNv2), Deep Neural Network (DNN), and Entire Space Modeling (ESMM).

their own input. Inspired by the LHUC, Parameter Personalized Network (PPNet) [4] introduces a gating mechanism called Gate Neural Unit (GNU), which allows personalized prior information to be injected into the network. The GNU consists of two neural network layers, takes the prior information features (user/item/author-side features) as input and outputs scaling weights to adjust the DNN parameters.

2.2 Straight-Through Estimator

When we aim to partition all the features into two parts, using binary neurons is a straightforward approach. However, binary neurons encounter the problem of gradients being unbackpropagatable due to their discrete nature. Fortunately, Straight-Through Estimator (STE) [2] introduces a solution to resolve this problem by coping the gradient with respect to the stochastic output directly as an estimator of the gradient. STE is also widely used for other tasks [10, 16, 22, 23], which demonstrate the effectiveness of STE. Inspired by STE, Gumbel-Softmax [10] employs a differentiable soft sampling approximation to make the discrete decision-making process differentiable, thereby enabling gradient backpropagation. AMTL [22] generates a mask vector to mask the undesired dimensions for each embedding vector using STE. OptFusion [23] automates

the learning of fusion via STE, encompassing both the connection learning and the operation selection.

2.3 Multi-Task Learning for Conversion Rate Prediction

Multi-Task Learning (MTL) is a subfield of machine learning in which multiple tasks are simultaneously learned by a shared model. Such approaches offer advantages like improved data efficiency, reduced overfitting through shared representations, and fast learning by leveraging auxiliary information [5].

Due to the neglect of data sparse problem, the sample selection bias problem, and the sequential dependencies among multi-stage sub-tasks in conversion rate prediction task, general MTL models [3, 11, 15] suffer from suboptimal performance. Consequently, specialized models [7, 12, 20] for multi-stage conversion rate prediction have been proposed in recent years to address these limitations. ESMM [12] eliminates sample selection bias by modeling CVR over entire space, and eliminates data sparsity problem employing a feature representation transfer learning strategy. AITM [20] implements an Adaptive Information Transfer module to adaptively learn what and how much information to transfer for different conversion stages, and uses a penalty loss to ensure the former task

should have a higher end-to-end conversion probability than the latter for the same audience. ResFlow [7] is a lightweight multi-task learning framework that enables efficient cross-task information sharing via residual connections between corresponding layers of task networks.

3 Proposed Model

As Shown in Figure 2, our proposed APPNet model consists of Embedding Layer, Sequence Feature Modeling, Concatenate Layer, Automatic Feature Partitioning (AFP), PPNet, Deep Cross Network (DCNv2) [17], three layers of Deep Neural Network (DNN), and Entire Space Modeling (ESMM). The Automatic Feature Partitioning (AFP) module and Parameter Personalized Network (PPNet) module are the core components of APPNet, both are highlighted in yellow in Figure 2. The AFP module has two implementations: feature-wise AFP and bitwise AFP. We will elaborate on them in Section 3.1. We adopt the PPNet implementation proposed in PEPNet [4] and will detail it in section 3.2.

3.1 Automatic Feature Partitioning (AFP)

The overall design of Automatic Feature Partitioning is shown in Figure 3: First, a learnable feature weight matrix is initialized for all features. Then, in the forward propagation of training stage, the feature weights are binarized via an indicator function to generate the feature mask (if the feature weight is greater than or equal to the weight threshold hyper-parameter λ , its feature mask is set to 1; otherwise, its feature mask is set to 0). Features with mask=1 are feed to the PPNet, while features with mask=0 are feed to the DNN of pCTR and pCVR estimation task. While in the backward propagation of training stage, gradients are back-propagated to both the features and their corresponding feature weights. Since DNN module and PPNet module have entirely distinct input features, there is no gradient conflict issue. During the iterative training of the model, the AFP module continuously explore new features by updating and optimizing feature weights based on gradients, ultimately achieving optimal performance for the conversion rate tasks. Specifically, we implemented two versions of AFP: feature-wise and bit-wise. The feature-wise AFP partitions features at the feature field granularity, while the bit-wise AFP partitions each bit of the feature embeddings. Further details are provided in Section 3.1.1 and 3.1.2.

3.1.1 Feature-wise Automatic Feature Partitioning. We denote $e_i \in \mathbb{R}^d$ as the embedding of feature f_i where d is the dimension size of feature embedding, and $E \in \mathbb{R}^{1 \times Nd}$ as the embedding matrix of all features: $E = [e_1, \dots, e_i, \dots, e_N]$. N is the number of features.

First, we initialize the learnable feature weights matrix $Z \in \mathbb{R}^{1 \times N}$ with xavier normal initializer [8]. To prevent feature weights from exceeding the 0 to 1 range during training, we applied the sigmoid activation function to process the features:

$$Z' = \sigma(Z), Z' \in \mathbb{R}^{1 \times N} \quad (1)$$

where $\sigma(\cdot)$ is the sigmoid activation function.

Then, to divide the features into two parts, we binarized the feature weights using a threshold-based indicator function, resulting in a feature mask matrix M . M is calculated as follows:

$$M = \mathbb{I}(Z' \geq \lambda) \quad (2)$$

where λ is the feature weight threshold hyper-parameter and $\mathbb{I}(Z' \geq \lambda)$

is the indicator function: $\mathbb{I}(Z' \geq \lambda) = \begin{cases} 1 & \text{if } Z' \geq \lambda \\ 0 & \text{else} \end{cases}$. As λ increases,

fewer features are selected as inputs to PPNet leading to weaker parameter personalization strength but increases the performance of DNN. Conversely, the opposite allocation has the reverse effect. Therefore, λ is important for balancing parameter personalization performance and conversion prediction performance.

If we directly apply the feature mask M to the features (using features with $M_i = 1$ as input to PPNet and features with $M_i = 0$ as input to DNN), then during gradient backpropagation, only the feature weights in Z with initial values greater than λ would be updated. This prevents the model from exploring new feature partitioning strategies. To enable end-to-end training and enable the model to continuously explore new feature partitioning strategies, we adopt the STE[2] method. Specially, as illustrated in Equation (3), we use the binarized feature mask M during the forward propagation while employing the continuous-valued feature weights Z' during the backward propagation, ensuring that all feature weights can be updated:

$$Z'' = \text{Tile}(\odot(M - Z') + Z'), Z'' \in \mathbb{R}^{1 \times Nd} \quad (3)$$

where $\odot(\cdot)$ means without gradient backpropagation and this part will be ignored in backpropagation stage. Since the shapes of M and Z' are $\mathbb{R}^N$, while the shape of E is $\mathbb{R}^{1 \times Nd}$, it is necessary to repeat M and Z' to align with the dimensions of matrix E . The "Tile" operation in Equation (3) is perform this replication.

Finally, the selected features S for PPNet and other features O for DNN of conversion rate prediction tasks can be calculated as follows:

$$S = \text{LN}(E) \odot Z'' + \odot(\text{LN}(E) \odot (1 - Z'')) \quad (4)$$

$$O = \odot(\text{LN}(E) \odot Z'') + \text{LN}(E) \odot (1 - Z'') \quad (5)$$

where $\odot$ is hadamard product and LN is short for Layer Normalization. Through this design, all features are fully utilized by both PPNet and AFP. Moreover, the gradient backpropagation stopping strategy ensures that the same feature does not simultaneously receive gradients from different modules, thereby avoiding gradient conflicts.

3.1.2 Bit-wise Automatic Feature Partitioning. To enable finer-grained feature partitioning, we propose the bit-wise automatic feature partitioning method. Given the bit-wise feature weight matrix $\text{Bit_}Z \in \mathbb{R}^{1 \times Nd}$, the overall computational workflow is largely consistent with the feature-wise version, with the detailed computation process as follows:

$$\text{Bit_}Z' = \sigma(\text{Bit_}Z), \text{Bit_}Z' \in \mathbb{R}^{1 \times Nd} \quad (6)$$

$$\text{Bit_}M = \mathbb{I}(\text{Bit_}Z' \geq \lambda) \quad (7)$$

$$\text{Bit_}Z'' = \odot(\text{Bit_}M - \text{Bit_}Z') + \text{Bit_}Z' \quad (8)$$

$$\text{Bit_}S = \text{LN}(E) \odot \text{Bit_}Z'' + \odot(\text{LN}(E) \odot (1 - \text{Bit_}Z'')) \quad (9)$$

$$\text{Bit_}O = \odot(\text{LN}(E) \odot \text{Bit_}Z'') + \text{LN}(E) \odot (1 - \text{Bit_}Z'') \quad (10)$$

The main difference between feature-wise AFP and bit-wise AFP is that the feature-wise AFP partitions features at the feature field granularity while the bit-wise AFP partitions each bit of the feature

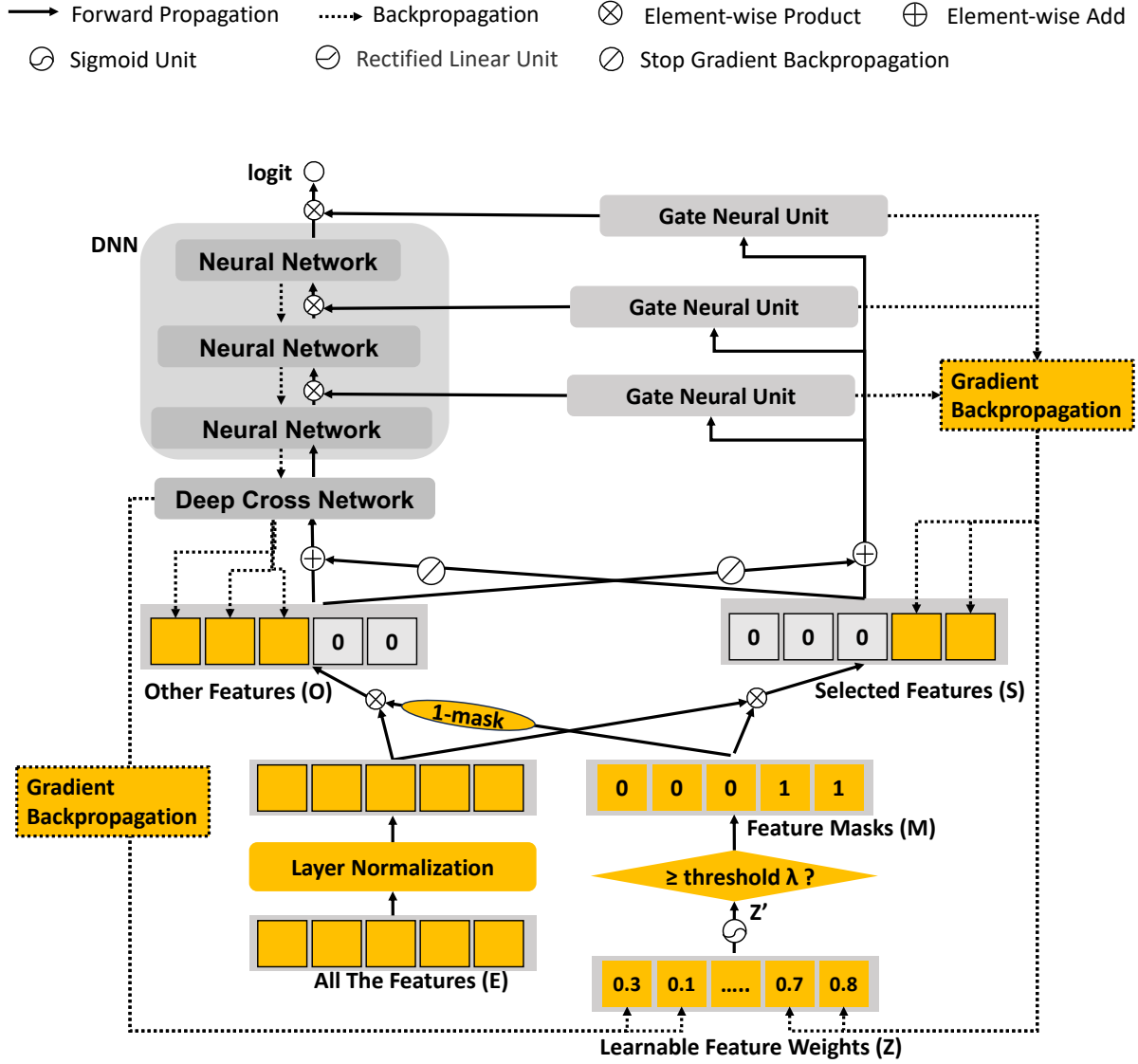


Figure 3: Automatic Feature Partitioning Module

embeddings. Notice the shape of $Bit_Z \in \mathbb{R}^{1 \times Nd}$ differs from the shape of $Z \in \mathbb{R}^{1 \times N}$.

3.2 Parameter Personalized Network (PPNet)

In this section, we detail the implementation of PPNet and explain how the feature-wise version of AFP interacts with it. Since the interaction between the bit-wise AFP and PPNet follows the same mechanism, we omit the redundant explanation here.

The PPNet consists of three layers of Gate Neural Unit (GNU). The GNU, followed the implementation of paper [4], consists of two neural network layers with ReLU as the first layer activation function and sigmoid as the second layer activation function. The output δ of GNU for layer l of DNN is calculated as follows:

$$S^{(l)} = \text{ReLU}(SW^{(l)} + b^{(l)}), l \in \{1, 2, 3\} \quad (11)$$

$$\delta^{(l)} = \gamma * \sigma(S^{(l)}W'^{(l)} + b'^{(l)}), l \in \{1, 2, 3\} \quad (12)$$

where σ is the sigmoid activation function, ReLU is short for Rectified Linear Unit. $W^{(l)}, b^{(l)}, W'^{(l)}, b'^{(l)}$ are learnable parameters. The γ is the scaling factor that is set as 2.

Given the output of gate neural unit and the other features O as DNN input, the prediction probability $\hat{y}$ of CTR or CVR can be calculated as follows:

$$O' = \text{DCN}(O) \quad (13)$$

$$H^{(1)} = \text{ReLU}(\text{NN}^{(1)}(O')) \quad (14)$$

$$H'^{(1)} = H^{(1)} \odot \delta^{(1)} \quad (15)$$

$$H^{(2)} = \text{ReLU}(\text{NN}^{(2)}(H'^{(1)})) \quad (16)$$

$$H'^{(2)} = H^{(2)} \odot \delta^{(2)} \quad (17)$$

$$\text{logit} = NN^{(3)}(H'^{(2)}) \odot \delta^{(3)} \quad (18)$$

$$\hat{y} = \sigma(\text{logit}) \quad (19)$$

where $DCN(*)$ is short for Deep Cross Network v2[17], $NN^{(l)}(*)$ is Neural Network of layer l .

3.3 Entire Space Multi-Task Loss Function

Given the label of CTR task y^{CTR} , the label of CVR Task y^{CVR} and their predictions $\hat{y}^{CTR}$ and $\hat{y}^{CVR}$, the loss of APPNet can be calculated as follows:

$$\text{Loss} = \mathcal{L}(y^{CTR}, \hat{y}^{CTR}) + \alpha \mathcal{L}(y^{CTR} \& y^{CVR}, \hat{y}^{CTR} \times \hat{y}^{CVR}) \quad (20)$$

where $\mathcal{L}(\text{label}, \text{prediction})$ is cross-entropy loss function, α is a hyper-parameter to balance the CTR loss and CTCVR loss, and

$$y^{CTR} \& y^{CVR} = \begin{cases} 1 & \text{if } y^{CTR} = 1 \text{ and } y^{CVR} = 1 \\ 0 & \text{else} \end{cases}$$

4 Experiments

Table 1: Statistics of the Offline Dataset

# User	# Item	# Impression	# Click	# Conversion
10,310,530	1,124,105	787,921,238	60,116,584	3,835,630

4.1 Dataset and Experimental Settings

Dataset Descriptions. We collect consumer logs from the product search system as the offline dataset. The statistics of the dataset are presented in Table 1. The dataset contains about 788 million training samples and 11 million test samples, which are divided by date.

Multi-Task Learning Backbone Models.

- **ESMM** [12] eliminates sample selection bias by modeling CVR over entire space, and eliminates data sparsity problem employing a feature representation transfer learning strategy.
- **AITM** [20] implements an Adaptive Information Transfer module to adaptively learn what and how much information to transfer for different conversion stages, and uses a penalty loss to ensure the former task should have a higher end-to-end conversion probability than the latter for the same audience.
- **ResFlow** [7] is a lightweight multi-task learning framework that enables efficient cross-task information sharing via residual connections between corresponding layers of task networks.

Parameter Personalization Method Competitors.

- **APG** [21] takes DNN input to generate the adaptive DNN parameters in an efficient and effective way by using over parameterization technology. In this paper, we adopt the over parameterization version of the APG implementation.
- **PPNet** [4] takes prior information features and other features (without gradient backpropagation) as input to compute personalized gate scores. These scores are then applied

via element-wise multiplication to the DNN's hidden units, thereby enabling personalized parameter modifications for the DNN. In this paper, following paper [4], we utilized user basic features, item basic features as prior information features, while other features were fed into the PPNet as inputs but without gradient backpropagation.

Implementation Details. When combining multi-task learning with parameter personalization methods, we aim to preserve the model architecture of the multi-task learning framework while applying personalization to the parameters of the DNN. Specifically, the combination is implemented as follows:

- **ESMM + APG/PPNet/APPNet:** The ESMM consists of pCTR DNN tower and pCVR DNN tower. All the parameter personalization methods generate or adjust parameters for both pCTR tower and pCVR tower.

$$pCTR = DNN_{pctr}(x) \quad (21)$$

$$pCVR = DNN_{pcvr}(x) \quad (22)$$

where x is the input features.

- **AITM + APG/PPNet/APPNet:** As illustrated in Equation (23)-(26), the AITM[20] consists of three core components for each task: Tower, Transfer and Adaptive Information Transfer(AIT). In this paper, all the parameter personalization methods generate or adjust parameters only for the Towers.

$$Tower_{k+1} = DNN_{k+1}(x) \quad (23)$$

$$Transfer_{k+1} = Transfer(AIT_k) \quad (24)$$

$$AIT_{k+1} = Attention(Transfer_{k+1}, Tower_{k+1}) \quad (25)$$

$$\hat{y}_{k+1} = \sigma(MLP(AIT_{k+1})) \quad (26)$$

where k means the task k , and σ is sigmoid activation and $\hat{y}_{k+1}$ is the prediction probability of task $k + 1$.

- **ResFlow + APG/PPNet/APPNet:** Same with ESMM, all the parameter personalization methods generate or adjust parameters for pCTR tower and pCVR tower.

Hyper-Parameter Tuning. When integrating a multi-task learning (MTL) backbone model with parameter personalization (PP) methods, we first optimize the hyper-parameters of the MTL model to achieve its optimal performance, and subsequently tune the hyper-parameters of the PP methods.

- **ESMM:** The hyper-parameter α in ESMM loss function (Equation (27)) is subjected to grid search within the ranges [0.5, 1.5, 2, 2.5, 3, 3.5, 4]. We set $\alpha = 2$ in this paper based on grid search.

$$\text{Loss} = \text{Loss}^{CTR} + \alpha \text{Loss}^{CTCVR} \quad (27)$$

- **AITM:** The behavioral expectation calibrator loss weight hyper-parameter β of AITM is subjected to grid search within the ranges [0, 0.2, 0.4, 0.6, 0.8, 1]. We set $\beta = 0.2$ in this paper based on grid search.

$$L(\theta) = \text{Loss}^{CrossEntropy}(\theta) + \beta \text{Loss}^{Calibrator}(\theta) \quad (28)$$

- **ResFlow:** In our scene, there are two tasks: CTR prediction and CTCVR prediction. Therefore, the loss weight hyper-parameter of tasks in ResFlow is same with ESMM. We conduct grid search to explore loss weight of CTCVR task with

Table 2: Comparison of models on the offline industry dataset. Metrics of CTR/CTCVR AUC/GAUC are presented. "-" means no parameter personalization Method is used. The "Feature-wise APPNet" means the combination of Feature-wise AFP and PPNet. The "Bit-wise APPNet" is the combination of Bit-wise AFP and PPNet. The best results among different parameter personalization models of each backbone model are indicated in bold and the second best are indicated in underline. The Gain represents the statistically significant ($p < 0.05$) improvement compare to the baseline backbone model.

Backbone Models	Parameter Personalization Method	AUC		GAUC	
		CTR (Gain)	CTCVR (Gain)	CTR (Gain)	CTCVR (Gain)
ESMM	-(baseline)	0.7604 (+0.0000)	0.8985 (+0.0000)	0.7101 (+0.0000)	0.8014 (+0.0000)
	APG	0.7587 (-0.0017)	0.8971 (-0.0014)	0.7048 (-0.0053)	0.7978 (-0.0036)
	PPNet	0.7637 (+0.0033)	0.9004 (+0.0019)	0.7092 (-0.0009)	0.8018 (+0.0004)
	Feature-wise APPNet (Ours)	<u>0.7655</u> (+0.0051)	0.9019 (+0.0034)	<u>0.7114</u> (+0.0013)	<u>0.8035</u> (+0.0021)
	Bit-wise APPNet (Ours)	0.7657 (+0.0053)	<u>0.9017</u> (+0.0032)	0.7124 (+0.0023)	0.8042 (+0.0028)
AITM	-(baseline)	0.7642 (+0.0000)	0.8930 (+0.0000)	0.7092 (+0.0000)	0.7979 (+0.0000)
	APG	0.7565 (-0.0077)	0.8955 (+0.0025)	0.7015 (-0.0077)	0.7954 (-0.0025)
	PPNet	0.7619 (-0.0023)	0.8943 (+0.0013)	0.7069 (-0.0023)	0.7964 (-0.0015)
	Feature-wise APPNet (Ours)	<u>0.7642</u> (+0.0000)	0.9005 (+0.0075)	<u>0.7104</u> (+0.0012)	<u>0.8019</u> (+0.0040)
	Bit-wise APPNet (Ours)	0.7651 (+0.0009)	<u>0.9004</u> (+0.0074)	0.7108 (+0.0016)	0.8027 (+0.0048)
ResFlow	-(baseline)	0.7644 (+0.0000)	0.9009 (+0.0000)	0.7095 (+0.0000)	0.8017 (+0.0000)
	APG	0.7585 (-0.0059)	0.8967 (-0.0042)	0.7043 (-0.0052)	0.7969 (-0.0048)
	PPNet	0.7644 (+0.0000)	0.9007 (-0.0002)	0.7098 (+0.0003)	0.8012 (-0.0005)
	Feature-wise APPNet (Ours)	0.7649 (+0.0005)	0.9017 (+0.0008)	<u>0.7111</u> (+0.0016)	0.8037 (+0.0020)
	Bit-wise APPNet (Ours)	0.7649 (+0.0005)	<u>0.9016</u> (+0.0007)	0.7114 (+0.0019)	0.8037 (+0.0020)

ranges [0.5, 1.5, 2.0, 2.5, 3, 3.5, 4]. We set CTCVR loss weight to 2.0 in this paper based on grid search.

- **APG:** The hyper-parameters P and K for over parameterization of APG are subjected to grid search within the ranges [256, 128, 64, 32, 16, 8, 4]. We set $P = 64$ and $k = 4$ in this paper based on grid search.
- **APPNet:** The feature weight threshold hyper-parameter λ in Equation (2) and (7) of APPNet is subjected to grid search within the ranges [0.4, 0.5, 0.6, 0.7, 0.8, 0.9]. We set $\lambda = 0.7$ in this paper based on grid search.

Shared Experimental Settings. To make a fair comparison, all the baseline models and our proposed model share some network structure and hyper-parameters, where we i) share the same feature information; ii) use the deep&cross network [17] to conduct feature interaction and a three-layer fully connected network with sizes of 512, 256, and 128 to implement the CTR/CVR towers; iii) use the ADAM optimizer with a fixed learning rate of 0.001 and a batch size of 2048 for the training process; and iv) apply L_2 regularization with a weight of $1e-5$, as well as batch normalization with a dropout rate of 0.3 to prevent over-fitting.

Metrics. We employ two widely used metrics to measure the offline performance of the models [11, 12, 15, 19, 24, 25]: Area Under Curve (AUC) [13] and Group Area Under Curve (GAUC) [26].

- **AUC:** *Area Under Curve* (AUC) is a widely used metric to measure the ranking result for the whole model. It indicates the probability that positive samples rank higher than negative samples.
- **GAUC:** *Group Area Under Curve* (GAUC) partitions test samples into groups via the consumer id, and AUC is calculated

by each group. It is defined as follows:

$$GAUC = \frac{\sum_u w_u \times AUC_u}{\sum_u w_u}, \quad (29)$$

where w_u is the sample size of consumer u .

Table 3: Comparison of the performance of APPNet with and without Layer Normalization (LN)

Models	AUC		GAUC	
	CTR	CTCVR	CTR	CTCVR
Feature-wise APPNet w/o LN	0.7639	0.9007	0.7090	0.8017
Feature-wise APPNet w/ LN	0.7661	0.9019	0.7118	0.8035
Improvements	+0.0022	+0.0012	+0.0028	+0.0018

Models	AUC		GAUC	
	CTR	CTCVR	CTR	CTCVR
Bit-wise APPNet w/o LN	0.7620	0.8989	0.7093	0.8015
Bit-wise APPNet w/ LN	0.7657	0.9017	0.7124	0.8042
Improvements	+0.0037	+0.0028	+0.0031	+0.0027

4.2 Offline Comparison Results

To demonstrate the generalization and effectiveness of APPNet, we inject APPNet and other parameter-personalized competitor models into different multi-task learning models for conversion rate prediction.

As shown in Table 2, on our offline dataset, our proposed Feature-wise APPNet achieved the best CTCVR AUC performance across three multi-task conversion rate prediction backbone models. The

Bit-wise APPNet ranked second in CTCVR UC, while it outperformed all others in CTCVR GAUC, with the Feature-wise APPNet following closely as the second-best performer.

The APG consistently underperformed compared to non-parameterized baselines across all three backbone architectures.

The PPNet with ESMM as the backbone improved CTCVR AUC by 0.0019 and CTCVR GAUC by 0.0004 compared to ESMM. However, when using AITM and Resflow as backbone models, the PPNet exhibited a decline in CTCVR GAUC performance compared to the backbone models.

Table 4: Performance of Bit-wise APPNet with different feature weight threshold λ

Threshold λ	AUC		GAUC	
	CTR	CTCVR	CTR	CTCVR
0.4	0.7630	0.9003	0.7096	0.8017
0.5	0.7654	0.9016	0.7113	0.8036
0.6	0.7660	0.9021	0.7119	0.8041
0.7	0.7657	0.9017	0.7124	0.8042
0.8	0.7663	0.9022	0.7124	0.8039
0.9	0.7656	0.9016	0.7114	0.8033

4.3 Influence of Layer Normalization

During DNN training, "covariate shift" [9] may occur as training progresses, leading to vanishing or exploding gradients and unstable training. When we scaling network parameters using personalized weights, this issue can be further exacerbated. Layer Normalization (LN) [1] was proposed to alleviate the optimization challenges in recurrent neural networks. MaskNet [18] also emphasized the importance of LN when introducing multiplicative interactions, indicating that LN is a best practice when performing multiplicative operations on DNN hidden layer parameters. To demonstrate the effectiveness of LN in AFP, as shown in Figure 3 and Equation (4)(5)(9)(10), we conduct this ablation study experiment. As shown in Table 3, both feature-wise AppNet and bit-wise AppNet achieved significant performance improvements after applying LN.

4.4 Influence of the Feature Weight Threshold Hyper-parameter λ

The hyper-parameter λ is used to select proper features for instance grouping to personalize DNN parameters. As λ increases, more features are selected as inputs to PPNet leading to stronger parameter personalization degree but reduces the performance of DNN. As λ decreases, fewer features are selected as inputs to PPNet leading to weaker parameter personalization strength but increases the performance of DNN. Therefore, the model's entire performance is sensitive to the choice of λ . To explore the influence of λ , we conducted grid search experiments. As shown in Table 4, taking ESMM-based bit-wise APPNet model as an example, as λ increases from 0.4 to 0.6, CTCVR AUC/GAUC of rises, peaks between 0.6 and 0.8, and declines beyond 0.8.

4.5 Case Study: Features Selected by Automatic Feature Partitioning Module in Feature-wise APPNet

We conducted a case study on the input features selected for parameter personalization in the Feature-wise APPNet model with ESMM as its backbone. As shown in Table 5, the first column lists the feature categories, the second column indicates the number of features in each category, the third column shows the input features selected by the automatic feature partitioning module for the gate neural unit, and the fourth column lists the top 3 features with the highest feature weights among the selected features.

From Table 5, we can find that the features automatically selected by the automatic feature partitioning module are quite different from prior-knowledge-based features (user/item-side information): only 3 out of 25 user basic features are selected, and 15 out of 54 item basic features are selected; While 27 item statistical features, 25 user-item cross features and some features from other categories are also selected.

4.6 Online A/B Test

To validate the performance of our model in the production environment, we chose to deploy the Bit-wise AppNet with ESMM as its backbone to the online system. This decision was made because it achieved the best CTCVR GAUC in offline experiments. We conduct online A/B test in the product search system within 7 days of June 2025. Figure 4 illustrates the online performance of Bit-wise APPNet compared to our online baseline model with ESMM as backbone model. The mean CTCVR has increased by 1.09%. Our online baseline model is the model shown in Figure 2 removing AFP module and PPNet module.

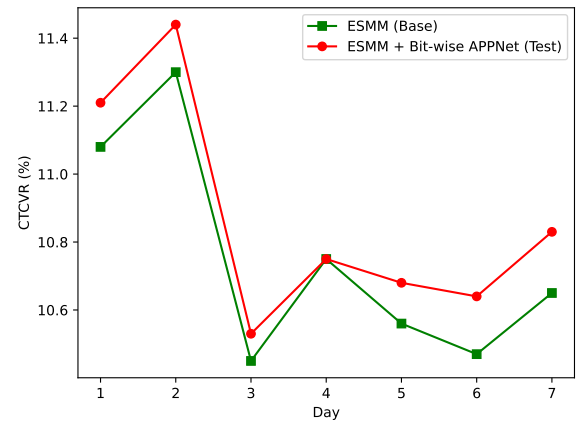


Figure 4: Online Performance of the proposed Bitwise APPNet

5 Conclusion

In this paper, we investigate the challenges of existing parameter personalization methods: taking prior information features as

Table 5: Features selected as input of Parameter Personalization Network in Feature-wise APPNet

Feature Category	# Features	# Selected Features	Top3 Selected Features
User Basic Features	25	3	purchase_power_level is_college_student has_children
User Historical Behavior Features	106	11	click_count_in_1day order_count_in_1day click_count_in_30days
Query Features	37	5	search_day_of_week search_keyword_type search_city
Item Basic Features	54	15	image_aesthetic_score category is_support_credit_pay
Item Statistical Features	150	27	rate_score rate_num pctr_in_7days
User-Item Cross Features	66	25	combo_user_age_item_brand combo_user_gender_item_brand combo_user_purchase_power_level_item_price

grouping condition leads to suboptimal performance, and using all the features for both parameter generation module and DNN causes gradient conflicts. To address the challenges, we proposed a novel APPNet model that uses the automatic feature partitioning module to select appropriate features to group instances, then uses PPNet to adjust DNN parameters of conversion rate prediction tasks for different groups, ultimately enhancing the model’s performance to characterize instances with different distributions. Extensive offline experiments demonstrate APPNet outperforms previous parameter personalized models. And online A/B testing in production system achieved a 1.09% improvement on conversion rate, validating its practical effectiveness. Furthermore, the proposed APPNet is an efficient, light-weight and plug-and-play method that can be easily injected into any DNN models.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* (2013).
- [3] Rich Caruana. 1997. Multitask learning. *Machine learning* 28, 1 (1997), 41–75.
- [4] Jianxin Chang, Chenbin Zhang, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. Pepnet: Parameter and embedding personalized network for infusing with personalized prior information. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3795–3804.
- [5] Michael Crawshaw. 2020. Multi-task learning with deep neural networks: A survey. *arXiv preprint arXiv:2009.09796* (2020).
- [6] Shangfeng Dai, Haobin Lin, Zhichen Zhao, Jianying Lin, Honghuan Wu, Zhe Wang, Sen Yang, and Ji Liu. 2021. POSO: personalized cold start modules for large-scale recommender systems. *arXiv preprint arXiv:2108.04690* (2021).
- [7] Cong Fu, Kun Wang, Jiahua Wu, Yizhou Chen, Guangda Huzhang, Yabo Ni, Anxiang Zeng, and Zhiming Zhou. 2024. Residual Multi-Task Learner for Applied Ranking. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4974–4985.
- [8] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 249–256.
- [9] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*. pmlr, 448–456.
- [10] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144* (2016).
- [11] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1930–1939.
- [12] Xiao Ma, Liqin Zhao, Guan Huang, Zhi Wang, Zelin Hu, Xiaoqiang Zhu, and Kun Gai. 2018. Entire space multi-task model: An effective approach for estimating post-click conversion rate. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 1137–1140.
- [13] Joel Myerson, Leonard Green, and Missaka Warusawitharana. 2001. Area under the curve as a measure of discounting. *Journal of the experimental analysis of behavior* 76, 2 (2001), 235–243.
- [14] Paweł Świetojanski, Jinyu Li, and Steve Renals. 2016. Learning hidden unit contributions for unsupervised acoustic model adaptation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 24, 8 (2016), 1450–1463.
- [15] Hongyan Tang, Junning Liu, Ming Zhao, and Xudong Gong. 2020. Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 269–278.
- [16] Aaron Van Den Oord, Oriol Vinyals, et al. 2017. Neural discrete representation learning. *Advances in neural information processing systems* 30 (2017).
- [17] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*. 1785–1797.
- [18] Zhiqiang Wang, Qingyun She, and Junlin Zhang. 2021. Masknet: Introducing feature-wise multiplication to CTR ranking models by instance-guided mask. *arXiv preprint arXiv:2102.07619* (2021).
- [19] Hong Wen, Jing Zhang, Yuan Wang, Fuyu Lv, Wentian Bao, Quan Lin, and Keping Yang. 2020. Entire space multi-task modeling via post-click behavior decomposition for conversion rate prediction. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 2377–2386.
- [20] Dongbo Xi, Zhen Chen, Peng Yan, Yinger Zhang, Yongchun Zhu, Fuzhen Zhuang, and Yu Chen. 2021. Modeling the sequential dependence among audience multi-step conversions with multi-task learning in targeted display advertising. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3745–3755.
- [21] Bencheng Yan, Pengjie Wang, Kai Zhang, Feng Li, Hongbo Deng, Jian Xu, and Bo Zheng. 2022. App: Adaptive parameter generation network for click-through rate prediction. *Advances in Neural Information Processing Systems* 35 (2022), 24740–24752.

- [22] Bencheng Yan, Pengjie Wang, Kai Zhang, Wei Lin, Kuang-Chih Lee, Jian Xu, and Bo Zheng. 2021. Learning effective and efficient embedding via an adaptively-masked twins-based layer. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 3568–3572.
- [23] Kexin Zhang, Fuyuan Lyu, Xing Tang, Dugang Liu, Chen Ma, Kaize Ding, Xiquang He, and Xue Liu. 2024. Fusion Matters: Learning Fusion in Deep Click-through Rate Prediction Models. *arXiv preprint arXiv:2411.15731* (2024).
- [24] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.
- [25] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
- [26] Han Zhu, Junqi Jin, Chang Tan, Fei Pan, Yifan Zeng, Han Li, and Kun Gai. 2017. Optimized cost per click in taobao display advertising. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2191–2200.